

# Clase 007 — Bash scripting para tareas de seguridad

Parte: **0 — Fundamentos y prerequisites** · Fuente: *GNU Bash Reference Manual*  Duración estimada: **110 min** · Nivel: **Fundamentos**

## Objetivo

Automatizar tareas repetitivas de seguridad con scripts de Bash robustos: barridos de red, parsing de resultados, comprobaciones de *hardening* y orquestación de herramientas. Al terminar escribirás scripts con variables, condicionales, bucles, funciones y manejo de errores.

## Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Estructurar** un script con shebang, variables y funciones.
2. **Usar** condicionales, bucles y `case` para lógica de control.
3. **Manejar** argumentos, entrada de usuario y códigos de salida.
4. **Aplicar** buenas prácticas de robustez ( `set -euo pipefail` , comillas).
5. **Automatizar** una tarea real de seguridad de principio a fin.

## Temas

| # | Tema                                         | Por qué importa                                   |
|---|----------------------------------------------|---------------------------------------------------|
| 1 | Shebang y ejecución                          | Cómo se interpreta y lanza un script              |
| 2 | Variables y expansión                        | Datos y sustitución de comandos                   |
| 3 | Condicionales <code>[[ ]]</code>             | Tomar decisiones según condiciones                |
| 4 | Bucles <code>for</code> / <code>while</code> | Iterar sobre hosts, puertos, líneas               |
| 5 | Funciones                                    | Reutilizar y organizar el código                  |
| 6 | Argumentos y <code>getopts</code>            | Scripts parametrizables                           |
| 7 | Robustez                                     | <code>set -euo pipefail</code> , quoting, trampas |
| 8 | Códigos de salida                            | Encadenar scripts y detectar fallos               |

## Definiciones y características

- **Shebang** ( `#!/usr/bin/env bash` ): primera línea que indica el intérprete. Clave: usa `env` para portabilidad.
- **Sustitución de comandos** ( `$( ... )` ): captura la salida de un comando en una variable. Clave: préfiérela a las backticks.

- **set -euo pipefail** : aborta ante errores, variables no definidas y fallos en pipes. Clave: convierte scripts frágiles en robustos.
- **Quoting**: entrecomillar variables ( `"$var"` ) evita *word splitting* y globbing. Clave: causa nº1 de bugs y de inyección.
- **Código de salida**: entero 0–255; 0 = éxito. Clave: `$?` lo lee; permite encadenar con `&&` / `||` .
- **Trap**: captura señales/eventos para limpiar ( `trap 'rm -f "$tmp"' EXIT` ). Clave: higiene ante interrupciones.

## Herramientas y preparación

Necesitas Bash (ya presente en Linux/Kali), un editor (nano, vim o VS Code) y **ShellCheck** para análisis estático:

```
sudo apt install shellcheck
```

Ten a mano herramientas que orquestarás en las prácticas: `ping` , `nc` (netcat), `nmap` (si está instalado). Trabaja siempre en tu laboratorio aislado.

## Laboratorio guiado

1. **Esqueleto robusto**. Crea `barrido.sh` :

```
bash #!/usr/bin/env bash set -euo pipefail red="${1:?Uso: $0 <prefijo /24, p.ej. 10.10.10>}"
```

2. **Bucle de descubrimiento** (ping sweep) en tu red interna:

```
bash for i in $(seq 1 254); do ip="${red}.${i}" if ping -c1 -W1 "$ip" &>/dev/null; then echo "[+] Activo: $ip" fi done
```

3. **Ejecuta** contra tu subred de laboratorio:

```
bash chmod +x barrido.sh ; ./barrido.sh 10.10.10
```

4. **Refactor a función**. Extrae la comprobación a `esta_activo()` y llama en el bucle.

5. **Guardar resultados** con marca de tiempo:

```
bash out="activos_$(date +%F_%H%M).txt"
```

y redirige los hallazgos con `>> "$out"` . 6. **Chequeo de hardening** simple: un script que verifique si el SSH permite login de root:

```
bash grep -qi "^PermitRootLogin yes" /etc/ssh/sshd_config \ && echo "[!] Root SSH habilitado" || echo "[ok] Root SSH restringido"
```

7. **Análisis estático**. Pasa ShellCheck y corrige avisos:

```
bash shellcheck barrido.sh
```

 **Nota ética**: los barridos y escaneos se ejecutan **solo** contra tu propio laboratorio o sistemas autorizados. Escanear redes ajenas puede ser ilegal.

## Ejercicios

1. Añade a `barrido.sh` la opción `-p PUERTO` con `getopts` para probar un puerto TCP con `nc -z`.
2. Escribe un script que reciba una lista de hosts por archivo y los recorra con `while read`.
3. Implementa manejo de errores: si falta `nmap`, avisa y termina con código distinto de 0.
4. Crea una función que valide que el argumento es una IP bien formada.
5. Usa `trap` para borrar un archivo temporal al salir, aun con Ctrl+C.
6. Escribe un mini auditor que revise 3 controles de hardening y devuelva un resumen con conteo de OK/FALLO.

## Reto verificable

Entrega un script `auditor.sh` parametrizable que reciba una subred, descubra hosts activos, para cada uno compruebe si tiene el puerto 22 o 80 abierto, y genere un informe con marca de tiempo. Debe pasar ShellCheck sin avisos y usar `set -euo pipefail`.

**Criterio de aceptación:** `shellcheck auditor.sh` no reporta problemas; ejecutado contra tu laboratorio produce un archivo de informe legible con hosts y puertos; y ante un argumento inválido termina con mensaje de uso y código de salida  $\neq 0$ .

## Errores comunes

| Síntoma / mensaje                           | Causa y cómo arreglar                                                                                                |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>unbound variable</code>               | Variable usada sin definir con <code>set -u</code> activo. Da valor por defecto ( <code>\${x:-}</code> ) o defínela. |
| El script "come" archivos con espacios      | Falta de comillas. Entrecomilla siempre <code>"\$var"</code> y usa <code>"\$@"</code> .                              |
| El pipe falla pero el script sigue          | Sin <code>pipefail</code> , solo cuenta el último comando. Añade <code>set -o pipefail</code> .                      |
| <code>[: ==: unary operator expected</code> | Comparación sin comillas y variable vacía. Usa <code>[[ ]]</code> y comillas.                                        |
| Permission denied al ejecutar               | Falta <code>chmod +x</code> o el shebang es incorrecto.                                                              |

## Preguntas frecuentes

**? ¿Bash o Python para automatizar seguridad?** Bash brilla pegando herramientas de línea de comandos y tareas rápidas del sistema. Para lógica compleja, estructuras de datos o red, Python (Clases 015–017) es mejor.

**? ¿Por qué `#!/usr/bin/env bash` y no `#!/bin/bash`?** `env` busca bash en el PATH, más portable entre distros donde bash puede estar en rutas distintas.

**? ¿ShellCheck es imprescindible?** Muy recomendable: detecta quoting, variables no usadas y bugs sutiles antes de que exploten. Intégralo en tu flujo.

? **¿Cómo depuro un script?** Ejecuta con `bash -x script.sh` para ver cada comando expandido, o añade `set -x` en la sección a inspeccionar.

## Referencias

---

- GNU Bash Reference Manual — <https://www.gnu.org/software/bash/manual/>
- ShellCheck — <https://www.shellcheck.net/>
- Google Shell Style Guide — <https://google.github.io/styleguide/shellguide.html>
- `man 1 bash`

## Siguiente clase

---

Clase 008 - Windows esencial para seguridad: arquitectura, registro y servicios